Good Evening Everyone :)

Starting at 9:05 pm

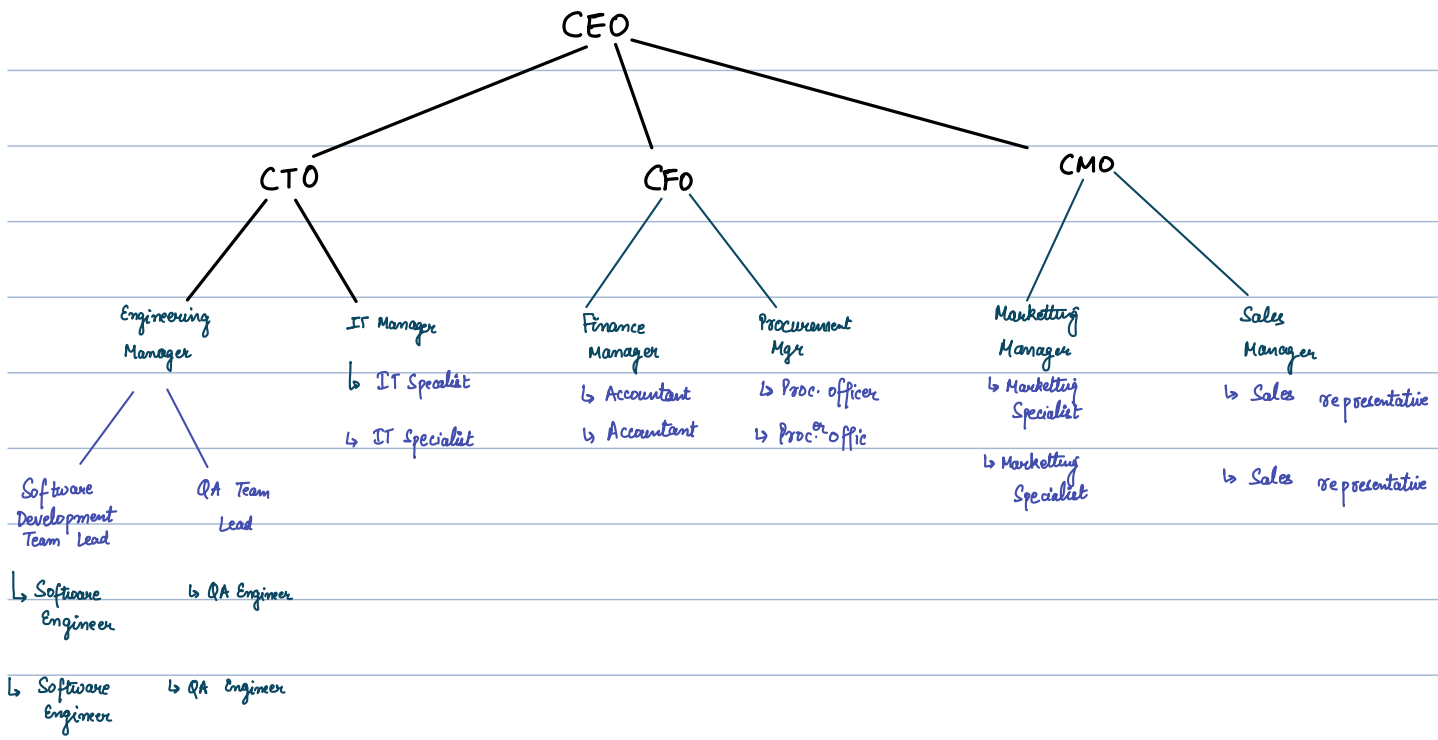
What is TREE ?

data structures like array, linked list, stack, queue are linear data structure

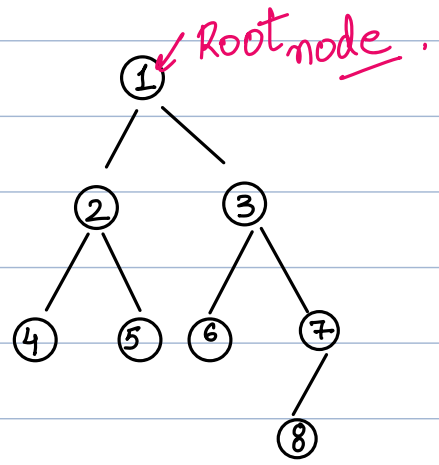
Non linear Data Structure

Tree is one example of Non linear data structure.

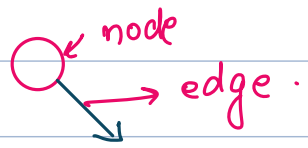
Example

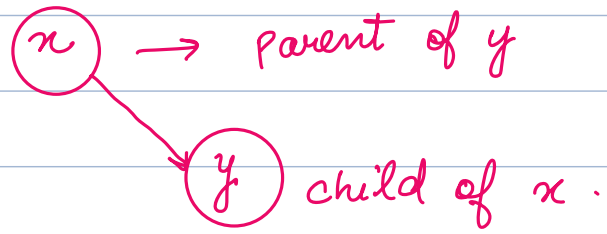


Tree Naming



Node: An element in a tree that contains data and may have child nodes connected to it





Root: The topmost node in a tree from which all nodes descend.

Parent

Child

Leaf. A node which is having no child is leaf node
4, 5, 6, 8 are leaf nodes.

Depth: The level at which one node reside in a tree
The root is at depth 0, . Node 2, 3 are at depth 1, & 4, 5, 6, 7 are at depth 2

Height The length of longest path from a leaf.
Height of root node is also known as height of tree

Subtree : A tree structure which is part of a given tree is Subtree

Siblings : Two nodes having Same parent
2 & 3, 4 & 5, 6 & 7

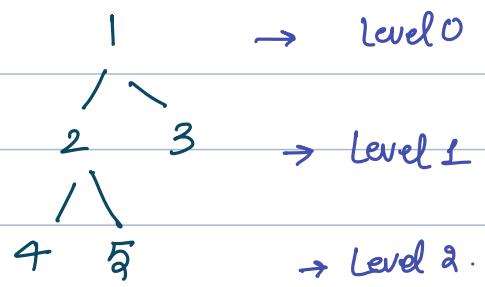
Ancestor

Descendant All nodes from child to leaf are the descendents

Quiz1. Can a leaf node also be a subtree?

Quiz2. Do all nodes have a parent?

Levels of a Tree



→ Level 0

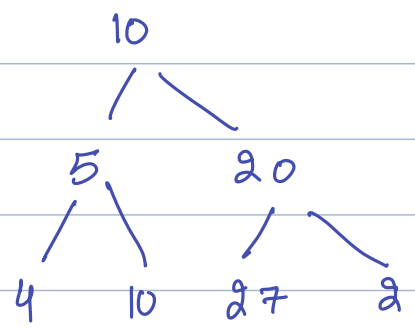
→ Level 1

→ Level 2.

Binary Tree

A type of tree in which each node can have at most 2 children

Every node can have 0, 1, 2 children.




```
Class TreeNode {
    int data
    TreeNode left
    TreeNode right
}
```

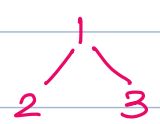
```
Class Node {
    int data
    Node next.
}
```

Traversal in a Tree

How many ways to traverse tree

L: Left SubTree , R: Right SubTree , N: Root Node.

L N R RLRNLRNN

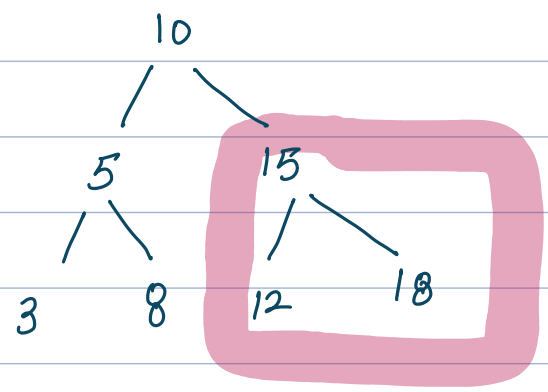


L N R → In Order ⇒ 2 1 3

N L R → Pre Order ⇒ 1 2 3

L R N → Post Order. ⇒ 2 3 1

Pre order



10 5 3 8 15 12 18

Pseudocode

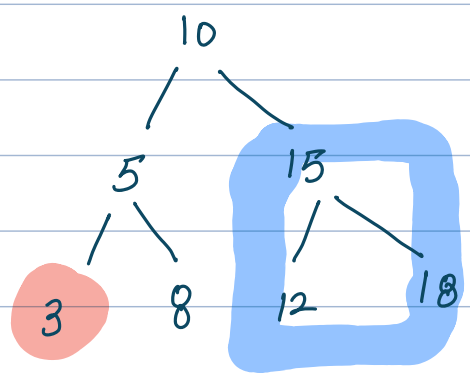
```
function preorder (root)
    if (root == null)
        return

    print (root.data)
    preorder (root.left)
    preorder (root.right)
}
```

IN-ORDER TRAVERSAL

1) Traverse left Tree

2) Visit the current node

3) Traverse the Right Tree



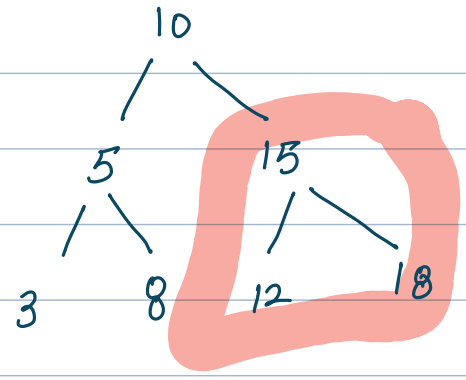
3 5 8 10 12 15 18

Pseudocode

```
function inorder (root) {

    if (root == null)
        return

    inorder (root.left)
    print (root.data)
    inorder (root.right)
}
```

Post-Order Traversal

L R N

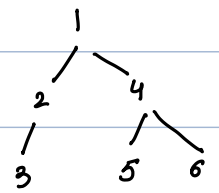


3 8 5 12 18 15 10

Pseudocode

```
function postorder (root)
    if (root == null)
        return

    postorder (root.left)
    postorder (root.right)
    print (root.data
}
```

Quiz 4.



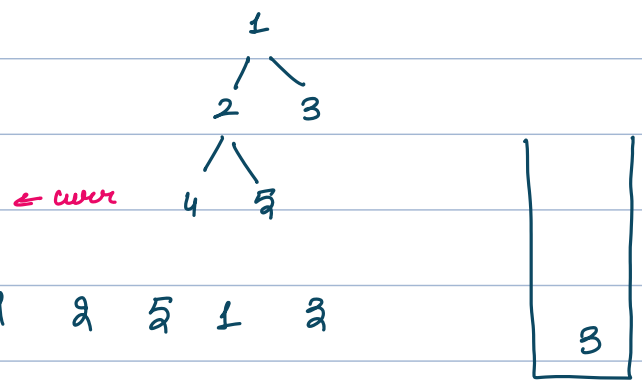
L N R

3 2 1 5 4 6

Iterative Inorder Traversal

Approach :-

1) Visit node in order, Left Subtree, current, right.



4 2 5 1 3

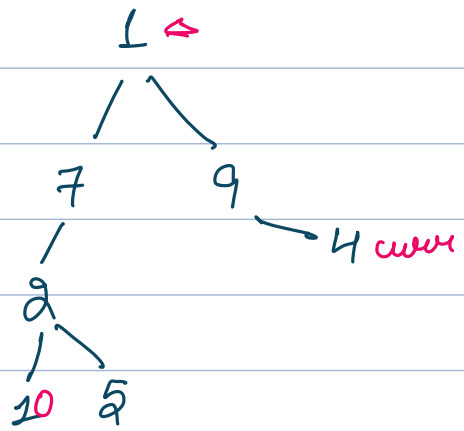
→ Pop 4 out and print
→ Pop 2 out and print
→ Push 5 on to the stack
→ Pop 5 out aand print
→ Pop 1 out and print
→ Push right child of 1 3 on stack
→ Pop 3 out and print

Pseudocode :

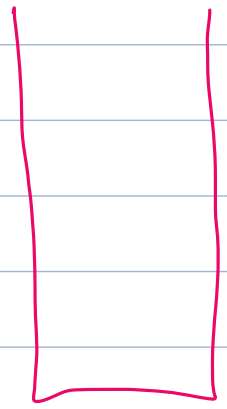
TC : O(N)
SC : O(N)

```
curr = root

while (curr != null || !st.isempty() ) {

        if (curr != null)
            st.push(curr)
            curr = curr.left)
        }
        else {
            curr = st.pop()
            print (curr.data)
            curr = curr.right)
        }
}
```

=> 10 2 5 7 1 9 4



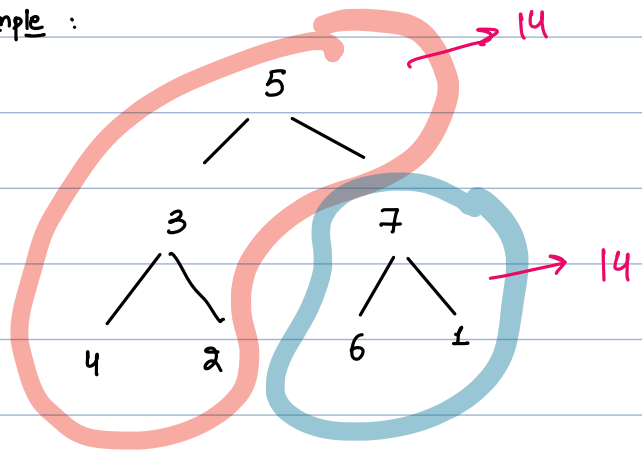
10 2 5 7 1 9 4



Equal Tree Partition

Given root of binary tree, return true if the tree can be split into two non-empty subtrees with equal sum. or false otherwise

Example :



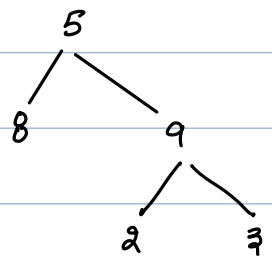
⇒ True.

Inorder - 8 5 2 9 3

Preorder - 5 8 9 2 3

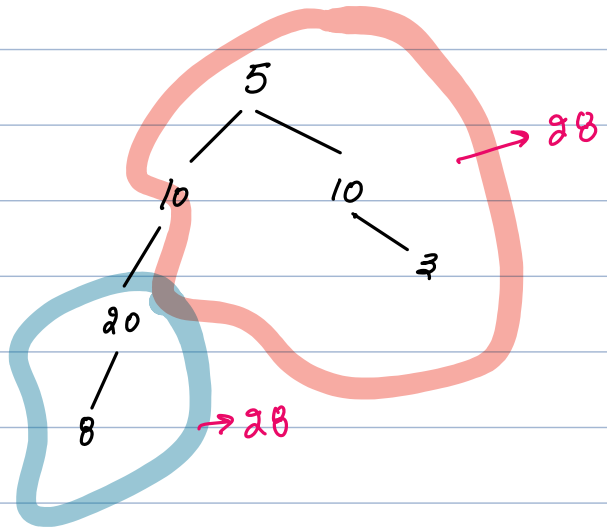
Postorder - 8 2 3 9 5

Example 2



⇒ false.

Quiz 5.



Yes.

Solution :

1. Total Sum Check :

If the total sum of all nodes in binary tree is odd, it is impossible to divide the tree into two subtrees with equal sums.

2. Subtree Sum Check

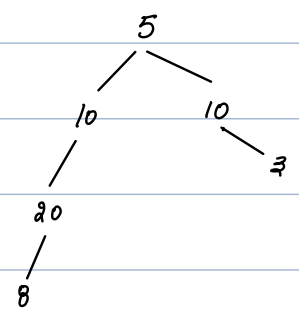
If we can find a subtree in binary tree with a sum equal to half of total sum, we can split into two subparts

This means that we don't necessarily need to compare sums of all possible subtrees, but we can look for a single subtree that meets the subtree sum check condition

PSEUDOCODE

```
function sum (TreeNode root) {
    if (!root) {
        return 0
    }
    return (sum (root.left) + sum (root.right) + root.val)
}
```

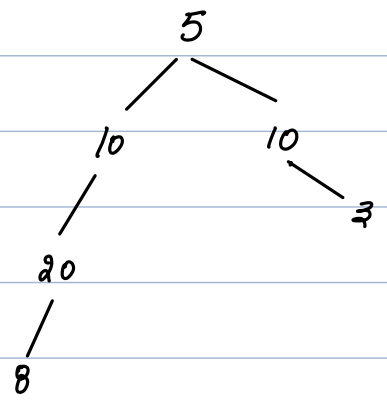
```
                                  10        26
function hasSubtreeWithHalfSum (TreeNode root, int totalSum) {
    if (!root)        return false

    int leftsum =     sum (root.left)
    int rightsum =    sum (root.right)
```



```
if (leftSum == totalSum/2 || rightSum == totalSum/2
    || hasSubTreeWithHalfSum (root.left, totalSum)
    || hasSubTreeWithHalfSum (root.right, totalSum) )
    return true
}
return false
}
```

TC: O(N)
SC: O(H).

```
function isEqualTreePartition ( TreeNode root) {
    if (! root)
        return false
    int totalSum = sum(root)          // 56
    if (totalSum % 2 == 1)
        return false
    return hasSubtreeWithHalfSum (root, totalSum)
}
```


5
10
10
20
3
8

?